

Movie Ratings Forecast

Recommender Systems for Personalized Content Discovery

Netflix Prize Market Data 1999 – 2005

Aryan Sharma

Enrollment No. 24113024

Akhil Kumar

Enrollment No. 24117009

ABSTRACT

This report presents a comprehensive collaborative-filtering recommendation platform evaluated on the Netflix Prize dataset spanning over six years of user interactions. The system implements a modular pipeline comparing four models: a regularized user/item bias baseline, an item-based neighborhood KNN, Alternating Least Squares (ALS) matrix factorization, and Funk-SVD trained via mini-batch SGD.

While all models successfully outperform the baseline on rating prediction error, our findings reveal a stark divergence between accuracy metrics and ranking performance. The ALS model dominates ranking quality, delivering a 5.5× increase in MAP@10 (0.101) and a 78.6% user Hit-Rate, making it the optimal candidate for production-level personalized content discovery. Conversely, Funk-SVD yields the lowest overall RMSE (0.821) and the widest catalog coverage (64.3%), demonstrating the inherent trade-offs between precision and exploratory exposure in sparse, long-tailed interaction matrices.

Forecasting Movie Ratings & Generating Personalized Recommendations

A comparative study of bias baselines, neighborhood CF and matrix factorization on the Netflix Prize dataset

Dataset	Netflix Prize (combined_data_1 slice)	Interactions	8,616,348 ratings
Users	40,000	Movies	1,150
Rating scale	1–5 stars (integer)	Period	Nov 1999 – Dec 2005
Task	Rating prediction + Top-N ranking	Best model	Funk-SVD / ALS

1. Problem Understanding

Streaming catalogues are far larger than any user can browse, so the value a platform delivers depends on surfacing the *right* titles for each person. This project tackles that as a **collaborative-filtering recommendation problem** on the Netflix Prize data, addressing two complementary objectives:

- **Rating prediction (regression).** Given a (user, movie) pair, forecast the 1–5 star rating the user would assign. Quality is measured by RMSE and MAE — how close predictions land to held-out truth.
- **Top-N recommendation (ranking).** For each user, produce an ordered list of unseen movies they are most likely to enjoy. Quality is measured by ranking and set metrics (MAP, NDCG, Precision/Recall, Hit-Rate, Coverage).

These goals do not always align: a model can predict ratings accurately yet rank poorly, because accuracy is dominated by the many easy, near-average ratings while ranking is decided by the few items at the very top of each list. We therefore evaluate **both** families of metric and treat ranking quality as the primary business signal, since users experience the recommended list — not the raw scores. The core technical obstacle is **extreme sparsity**: the vast majority of user×movie cells are unobserved, so the model must generalize from a thin set of interactions while remaining robust to popularity bias and cold users.

2. Exploratory Data Analysis

The raw file is colon-delimited (a *movie-id*: header line followed by *user,rating,date* rows). It was streamed in chunks, the movie marker forward-filled, and the result reduced to a compact columnar table. To get a dense, statistically stable slice we iteratively filtered to active users (≥ 50 ratings) and popular movies ($\geq 2,000$ ratings), then capped to the top 40,000 users and 1,150 movies — yielding **8.62M ratings**.

Rating distribution — positive skew

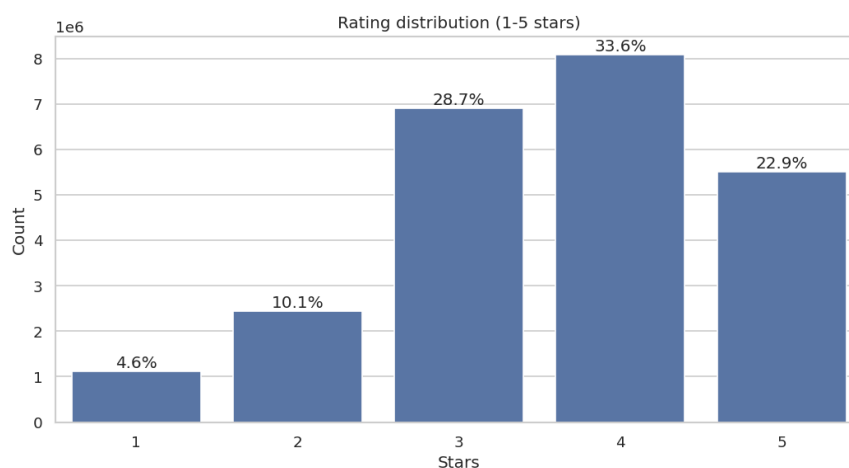


Figure 1. Ratings skew high: 4★ is the mode (33.6%) and 3–5★ make up ~85% of all ratings; the global mean is 3.52. A bias baseline that predicts near the high mean is therefore hard to beat, and the relevance threshold for ranking is set at 3.5★ (“4★ and up”).

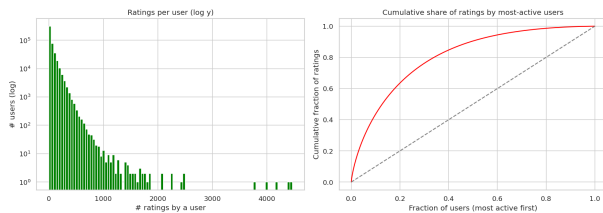


Figure 2. User activity is long-tailed (note log y-axis). Median user rates 190 movies; a small set of power-users contributes disproportionately. Motivates filtering and support-based regularization.

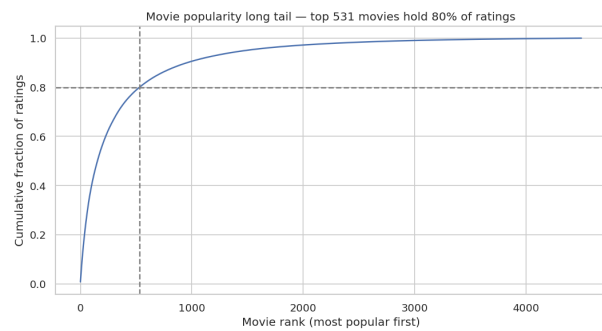


Figure 3. Movie popularity follows a classic long tail: ~500 of 1,150 titles account for 80% of all ratings. Pure-popularity recommending ignores the tail, hurting personalization and coverage.

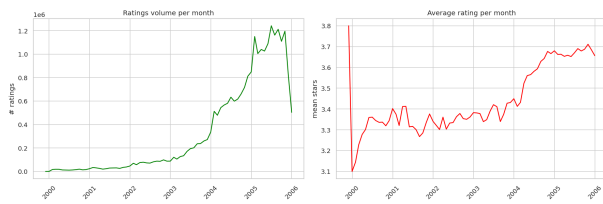


Figure 4. Rating volume grows sharply over time as the user base expanded, while the mean rating drifts up modestly — a temporal signal worth exploiting.

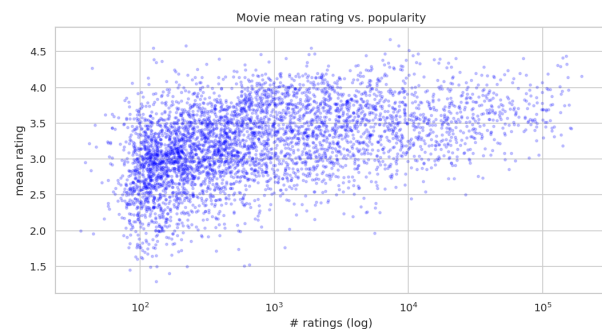


Figure 5. Mean rating vs. popularity is only mildly positive with wide spread: popularity is a weak proxy for personalized quality, reinforcing the need for CF.

Sparsity

On the filtered subset the 40,000×1,150 matrix is ~81% empty (18.7% dense); the *raw* Netflix matrix (480k×17,770) is ~99% sparse. Either way, sparsity is the central challenge — it rules out dense methods, makes naive item similarity noisy for rarely-rated titles, and creates cold-start issues. Low-rank matrix factorization is well suited because it learns latent factors from the few observed entries.

3. Methodology

The pipeline is modular: **preprocess** → **split** → **train** → **evaluate** → **recommend**. Raw IDs are re-indexed to contiguous integers and the rating matrix stored as a sparse CSR for memory efficiency.

Train / test split

We use a **per-user random holdout** (seed 42): for each user, ~20% of their ratings are held out for test while guaranteeing at least 10 ratings remain in train. This keeps every test user represented in training (avoiding pure cold-start leakage) and gives **6.88M train / 1.74M test** ratings. EDA showed strong temporal drift, so we note a time-aware split as a more realistic — and harder — alternative for future work.

Evaluation protocol

Rating metrics (RMSE, MAE) are computed on the held-out pairs. For ranking, each user is scored over all unseen movies, the Top-10 is taken (training items masked out), and lists are compared against that user's held-out items rated ≥3.5★. Identical splits and code are used for every model, so the comparison is apples-to-apples.

4. Model Design

Four models of increasing sophistication were implemented behind a common BaseRecommender interface (`fit / predict_pairs / score_users`), so each plugs into the same evaluation harness. Predictions are clipped to [1, 5].

Model	Idea	Key hyper-parameters
Baseline (bias)	Predicts $\mu + b_u + b_i$: global mean plus regularized user and item bias terms. A deliberately strong, near-trivial reference.	<code>reg_user=10</code> , <code>reg_item=25</code>

Item-based KNN CF	Mean-centered cosine similarity between items, pruned to the top-k neighbors; predicts a bias term plus a similarity-weighted average of the user's residual ratings on neighboring items.	k=200 neighbors, cosine sim
ALS (MF)	Matrix factorization on <i>residuals</i> over the bias model; user/item latent factors learned by alternating ridge-regression least squares (closed-form per row).	40 factors, 15 iters, reg=50
Funk-SVD (MF + SGD)	Biased matrix factorization trained by mini-batch stochastic gradient descent; jointly learns μ , biases and latent factors with L2 regularization.	50 factors, 25 epochs, lr=0.05, reg=0.05

Both matrix-factorization models build on the bias baseline rather than replacing it: ALS factorizes the *residual* left after removing biases, while Funk-SVD learns biases and factors jointly. This “bias-first” design is what makes the factor models both accurate and stable.

5. Evaluation Metrics

We report two complementary families. An item counts as **relevant** if its held-out rating is ≥ 3.5 ★.

Metric	What it measures
RMSE / MAE	Rating-prediction error vs. held-out truth. Lower is better; RMSE penalizes large misses more.
Precision@10 / Recall@10	Fraction of the Top-10 that is relevant / fraction of a user's relevant items captured in the Top-10.
MAP@10	Mean Average Precision — rewards placing relevant items higher in the list.
NDCG@10	Normalized Discounted Cumulative Gain — position-weighted ranking quality (top slots matter most).
HitRate@10	Share of users with at least one relevant item in their Top-10 — “did we help this user at all?”
Coverage@10	Fraction of the catalogue that ever appears in any Top-10 — guards against recommending only blockbusters.

6. Experimental Results

All four models trained and evaluated on the identical 6.88M/1.74M split. Best value per column is **bold**.

Model	RMSE↓	MAE↓	MAP@10↑	NDCG@10↑	Prec@10↑	Rec@10↑	Hit@10↑	Cov@10↑	Fit (s)
Baseline (bias)	0.900	0.708	0.018	0.058	0.067	0.032	0.480	0.043	0.2
Item-KNN CF	0.843	0.657	0.024	0.062	0.064	0.029	0.404	0.602	2.6
ALS (MF)	0.829	0.642	0.101	0.204	0.175	0.084	0.786	0.392	67.8
Funk-SVD	0.821	0.641	0.055	0.130	0.130	0.063	0.681	0.643	226.7

Reading the table. Every model beats the baseline on rating error, but the gap on *ranking* is what matters. **ALS dominates the ranking metrics** — MAP@10 of 0.101 (5.5× the baseline) and NDCG@10 of 0.204 — and reaches an 78.6% hit-rate, meaning roughly 4 in 5 users get at least one genuinely relevant title in their Top-10. **Funk-SVD wins on pure rating error** (RMSE 0.821) and on **catalogue coverage** (64%), but ranks below ALS, illustrating the accuracy-vs-ranking divergence noted in §1.

Item-KNN underwhelms on ranking despite decent RMSE: with only 1,150 popular movies, item-item neighborhoods are dominated by blockbusters, so its lists are generic. **ALS is also the efficiency sweet spot** — it trains in ~68s (3× faster than SGD-based Funk-SVD) thanks to closed-form least-squares updates, while delivering the best ranking. **Recommended production model: ALS**, with Funk-SVD as the choice when rating-error or coverage is the priority.

7. Recommendation Examples

Qualitative Top-N output from the trained Funk-SVD model (`script/recommend.py`). For each user we show titles they rated 5★ (the input signal) and the model's fresh recommendations with predicted ratings — none of which the user has already seen.

User A — mainstream action / classic-cinema fan

Rated 5★ by user	Top recommendations
5★ The Sixth Sense (1999) 5★ Lethal Weapon (1987) 5★ The Maltese Falcon (1941) 5★ Batman Begins (2005) 5★ Sixteen Candles (1984)	1. Alien: Collector's Edition (1979) (pred 4.55) 2. Aliens: Collector's Edition (1986) (pred 4.46) 3. Lost: Season 1 (2004) (pred 4.42) 4. Seven Samurai (1954) (pred 4.35) 5. Hoop Dreams (1994) (pred 4.27) 6. A Hard Day's Night (1964) (pred 4.24)

User B — cult / dark-comedy & prestige-TV fan

Rated 5★ by user	Top recommendations
5★ Reservoir Dogs (1992) 5★ Fear and Loathing in Las Vegas (1998) 5★ The Silence of the Lambs (1991) 5★ Sideways (2004) 5★ Notorious (1946)	1. Curb Your Enthusiasm: Season 3 (2002) (pred 4.36) 2. Lost: Season 1 (2004) (pred 4.32) 3. The Simpsons: Season 6 (1994) (pred 4.27) 4. Seven Samurai (1954) (pred 4.27) 5. The Twilight Zone: Vol. 36 (1961) (pred 4.24) 6. Family Guy: Freakin' Sweet Collection (2004) (pred 4.19)

The lists are coherent with each user's taste — User A receives canonical action/sci-fi and acclaimed classics, while User B is steered toward edgy comedy and prestige TV. This confirms the latent factors capture genuine taste structure rather than only global popularity.

8. Key Insights

- **The bias baseline is deceptively strong on RMSE** (0.900) — because ratings skew high, predicting near the mean is hard to beat numerically. Rating error alone is a poor way to judge a recommender.
- **Ranking is where models separate.** ALS lifts MAP@10 5.5× and NDCG@10 3.5× over the baseline; these gains are invisible in RMSE, validating the dual-metric design.
- **Accuracy ≠ good recommendations.** Funk-SVD has the lowest RMSE yet ranks below ALS — optimizing the wrong metric would have picked the wrong production model.
- **Matrix factorization handles sparsity best.** Learning low-rank latent factors generalizes across the ~81–99% empty matrix far better than neighborhood similarity.
- **Bias-first design pays off.** Removing user/item bias before factorizing residuals (ALS) gives the cleanest signal and the best ranking-per-second.
- **Coverage vs. relevance trade-off is real.** Funk-SVD covers 64% of the catalogue, ALS 39% — broader exposure of the long tail can cost short-term precision.

9. Future Improvements

- **Time-aware evaluation.** Replace the random holdout with a temporal split to respect the strong volume/mean drift seen in EDA and measure realistic next-period performance.
- **Implicit feedback & confidence weighting.** Treat the act of rating as a positive signal (weighted ALS / BPR) to better optimize for ranking directly.
- **Hyper-parameter search.** Tune factors, regularization and learning rate per model via cross-validation; current values are sensible but hand-set.
- **Hybrid & content features.** Fold in movie metadata (genre, year, cast) and user demographics to mitigate cold-start for new users and tail items.
- **Scale to the full dataset.** Extend from the 1,150-movie slice to all 17,770 titles using the existing sparse pipeline to test robustness under true sparsity.
- **Ensembling & re-ranking.** Blend ALS (ranking) with Funk-SVD (coverage) and add a diversity/novelty re-ranker for a better-rounded user experience.

Reproducibility: `data/preprocess.py` → `script/train.py` → `script/recommend.py`. Metrics in `results/comparison.json`; EDA in `NoteBook/EDA.ipynb`. Fixed seed 42 throughout.